



Analytics

Assignment One

Nandi Malinga

Topollo Mokhatla

23rd March 2026

Question 1

Cape Town is experiencing a pronounced housing affordability crisis, driven in part by rising short-term rental demand from digital nomads and the relative affordability of local assets due to a weakened Rand (Moodley et al., 2025). This has incentivised landlords to shift properties away from the long-term rental market, exerting upward pressure on rents (Moodley et al., 2025). These dynamics are particularly concerning on Airbnb, where hosts are not required to disclose the number of listings they operate, obscuring the extent of multi-property ownership and complicating efforts to assess market concentration. We therefore aim to distinguish between typical hosts and those operating disproportionately large portfolios using observable listing-level features, addressing the informational gaps created by this lack of platform transparency.

The response variable **highhost** was constructed by thresholding **host_listings_count**, classifying a host as “high” if they operate nine or more listings. This threshold is not arbitrary: the median host in the Cape Town Airbnb market holds three listings in the training data, so the cutoff corresponds to three times the median — defining a **highhost** as one operating at least three times as many listings as half the platform’s host population. Lowering the threshold would marginally improve class balance but at the cost of conflating low hosts with genuine **highhosts**, undermining the conceptual validity of **highhost** as a classification target. The chosen threshold therefore represents a deliberate trade-off between statistical and substantive considerations, yielding a class distribution of approximately 69.5%:30.5% (low:high) in the training data. While not perfectly balanced, this is sufficient enough to prevent trivial majority-class exploitation - as a naive classifier would achieve 69.5% Accuracy, and this constitutes the relevant baseline for model evaluation.

We selected ten covariates on substantive grounds. **host_since** was feature-engineered to retain only the year of registration, preserving tenure information while discarding redundant date-level granularity. **neighbourhood_ward** was preferred over raw latitude and longitude coordinates, as ward boundaries capture administratively meaningful demand-side variation more interpretably than continuous spatial coordinates. The remaining covariates were retained as plausible proxies for either the operational scale or commercial orientation of a host, as detailed below:

host_acceptance_rate: Reflects operational capacity and the extent to which hosting is managed commercially.

host_is_superhost: Awarded on sustained performance metrics; likely correlated with the systematic management characteristic of **highhosts**.

room_type: Entire property listings are more consistent with commercial operation than private or shared rooms.

accommodates: Larger capacity properties are more viable as commercial listings.

bedrooms: Further characterises listing scale, closely related to capacity.

price: Pricing strategy may differ systematically between casual and commercial hosts.

availability_30: Active short-term availability reflects consistent listing management, expected to be higher among **highhosts**.

number_of_reviews: Proxies listing activity and booking frequency, both elevated for commercially-operated listings.

Question 2

Having identified the restricted set of covariates, the remaining modelling procedure is largely data-driven, with explicit decision rules guiding selection.

We consider four model classes: Regularised Elastic-net Logistic Regression, Random Forest, Gradient Boosted Trees, and K-Nearest Neighbours. Within each class, hyperparameters are selected by maximising 10-fold cross-validation Accuracy (first decision rule), after which the four best models from each model class are ranked across five metrics: Accuracy, F1 score, Precision, Recall, and Specificity (second decision rule). Accuracy and Recall are afforded primacy in the final model selection given the asymmetric costs of misclassification in this context: since Airbnb does not publicly disclose host portfolio information, false negatives — failing to identify a **highhost** — are more costly than false positives, as they directly reproduce the platform opacity this analysis seeks to address.

These decision rules are but a guide, and may need adjustment as in some cases interpretability could be prioritised over predictive performance; resulting in a *slightly* worse performing model being preferred over the best.

a) Model Development

Regularised Elastic-net Logistic Regression

The elastic-net approach combines Ridge (L_2) and Lasso (L_1) regularisation, balancing coefficient shrinkage to reduce variance and retain correlated variables, while encouraging sparsity to eliminate less important predictors. This helps manage the high dimensionality of our dataset and improve interpretability.

We generate α values over $[0, 1]$ in 0.1 increments, and fit them to `cv.glmnet`, using a regularised logistic regression model and optimising for λ using 10-fold cross-validation. In particular, we care about two λ candidates per α : `lambda.min`, the λ minimising CV misclassification error, and `lambda.1se`, the largest λ within one standard error of that minimum.

The best α is chosen by selecting the model that has the greatest cross validation Accuracy at its `lambda.min` or `lambda.1se` value.

For our dataset, we selected `lambda.1se` to prioritise parsimony. Given the large number of predictors, this decision promotes a simpler and more interpretable model, while avoiding a substantial loss in Accuracy. The resulting model shrinks 37 feature coefficients to 0, and achieves a cross-validation Accuracy of 69.75%², with optimal hyperparameters $\alpha = 0.51$ and $\lambda_{1se} = 0.00996$. Predictions on the training data are summarised in Table 1. The elastic-net model’s cross-validation Accuracy only barely edges that of a naive classifier (predicting all observations as not **highhost**). The confusion matrix shows many false negatives, indicating a strong bias towards the majority class.

Table 1: Best Elastic-Net Model Confusion Matrix

		Observed	
		Not Highhost	Highhost
Predicted	Not Highhost	5499 (TN)	2355 (FN)
	Highhost	65 (FP)	81 (TP)

Note: TN = True Negative, FN = False Negative, FP = False Positive, TP = True Positive.

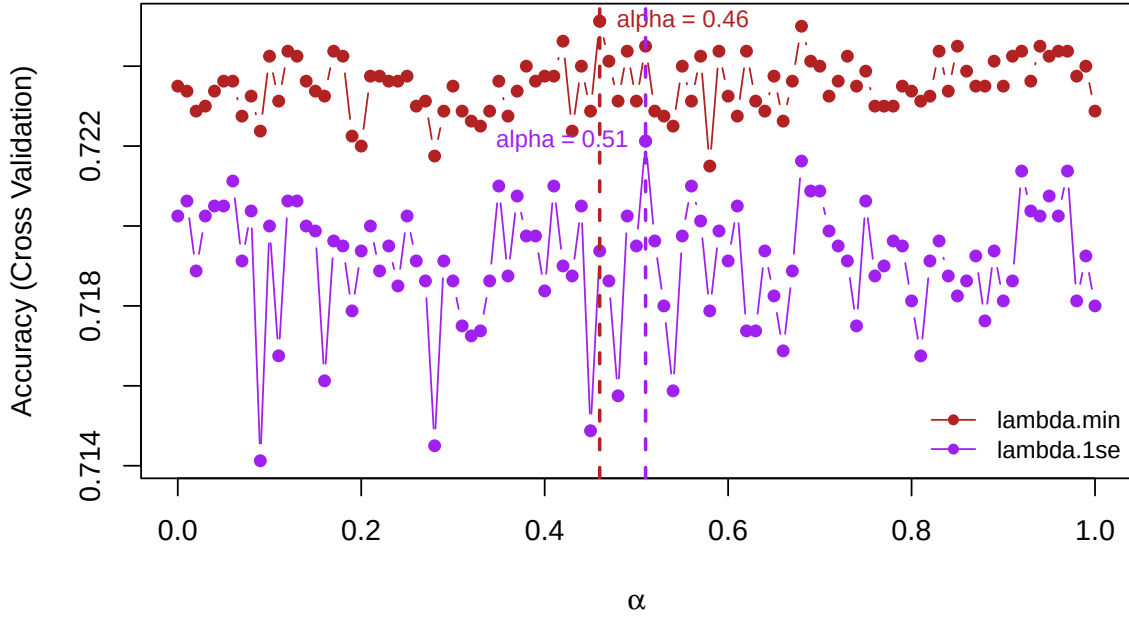


Figure 1: Elastic-net Accuracy (Cross-Validation) Across the Hyperparameter Space

Random Forest

The second model we applied was a random forest approach, which is a tree-based method designed to reduce the variability of predictions, particularly in datasets with correlated predictors. It involves bagging, which grows many decision trees on bootstrapped samples of the data. However, each tree only considers a random subset of predictors at each split. This reduces correlation between trees that occurs when a few strong features dominate early splits. By averaging predictions across all trees, the model reduces noise and produces more stable predictions.

This approach is well suited to our dataset due to the large number of features, as the combination of bootstrapping and random feature selection allows different trees to explore different subsets of the data and in the feature space while remaining robust. This encourages a wider range of covariates to contribute to the model and reduces correlation between trees, limiting the dominance of any single covariate. The averaging process helps prevent overfitting by smoothing noise from individual trees. Although random forests are often considered “black-box” algorithms, techniques such as variable importance plots can provide insight into the most influential covariates.

Our random forest model (`mod_randomforest`) was trained with 250 trees, and uses 10-fold cross-validation to tune two hyperparameters:

- `mtry` - the number of variables considered at each split (tested across all its possible values from 1 to 74), and
- `min.node.size` - the control for the depth of the tree (tested over values 1, 5 and 20).

As represented in Figure 2, the best model was achieved with `mtry` = 28 and `min.node.size` = 1, yielding a cross-validated Accuracy performance of 86.64%² and OOB prediction error of 9.99%.

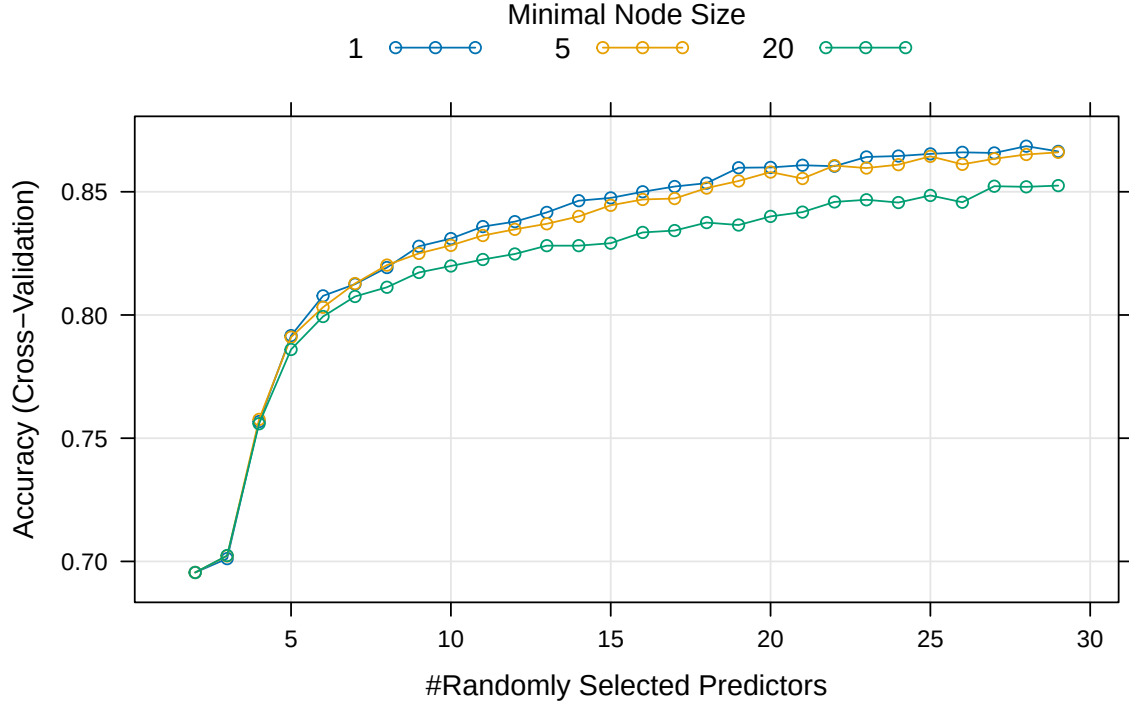


Figure 2: Random Forest Accuracy (Cross-Validation) Across the Hyperparameter Space

Gradient Boosted Model

Unlike random forests, which grow trees independently, boosting constructs trees sequentially. At each iteration, a small decision tree is fitted to the pseudo-residuals of the current model, representing the model’s errors at that stage. Each tree is then added to the model scaled by a learning rate λ , progressively correcting the errors of preceding iterations. For this data, the model effectively uses logistic regression to model probabilities under a Bernoulli loss function.

For our gradient boosted tree model (`mod_gbm`), we use 10-fold cross-validation to tune the following hyperparameters:

- **n.trees** - represents the number of sequential trees fitted to updated residuals, over the search space 5 000, 10 000, and 20 000. Increasing the number of trees reduces training error but may eventually lead to overfitting.
- **interaction.depth** - maximum tree depth (tested over values 1, 2, and 5). This determines the level of interaction between variables that can be captured.
- **shrinkage** - the learning rate (tested at values 0.1 and 0.01).

The high-dimensionality of our data is handled well by this approach. With 74 covariates in our dataset, GBM efficiently selects features by choosing tree splits that most reduce prediction error. As a result, only 42 covariates were found to have a non-zero influence on the response, and are retained across trees to explain the remaining structure in the residuals. Furthermore, by allowing multiple splits within each tree, the model can capture important interactions between variables without having to compute them directly. For example, interactions between `host_since_year` and `number_of_reviews` may influence `highhost` classification, since a more experienced host with a greater number of reviews may be considered more established and reliable.

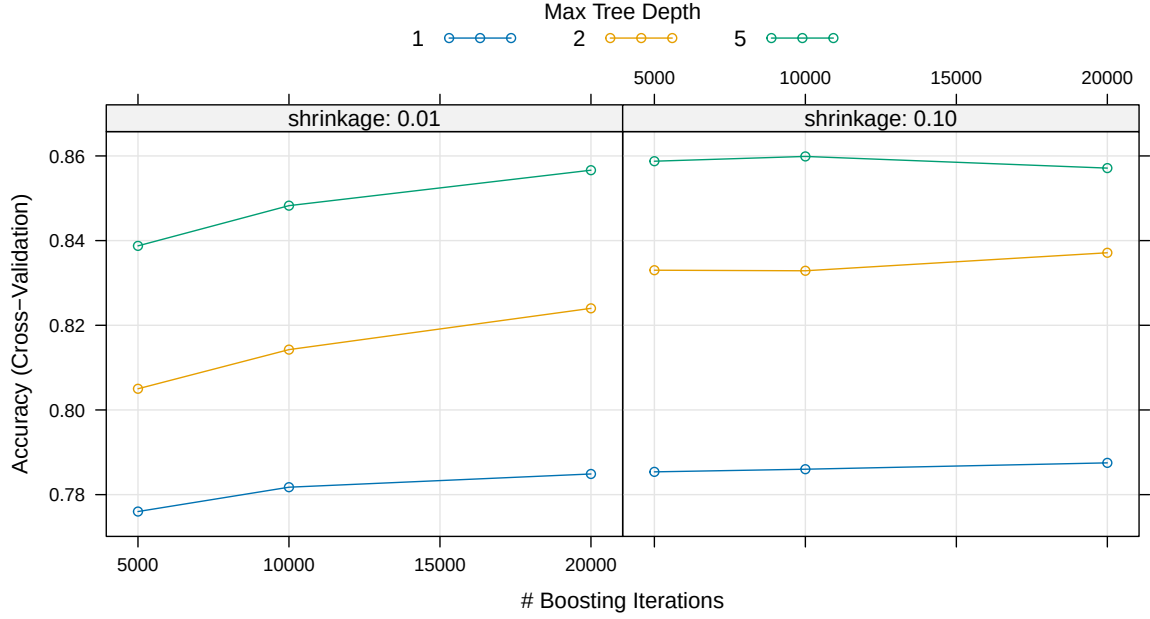


Figure 3: Gradient Boosted Model Accuracy (Cross-Validation) Across the Hyperparameter Space

From Figure 3, we observe that the best model selects `n.trees` = 10 000, with `shrinkage` = 0.1 and `interaction.depth` = 5, to obtain an Accuracy of 85.66%².

K-nearest neighbours (KNN)

Finally, it would be remiss not to consider the K-nearest neighbours (KNN) algorithm for this dataset, given its simplicity and surprisingly strong performance in a variety of contexts. The KNN algorithm operates by storing the training observations and predicting the class of an observation based on the majority class among its k nearest neighbours.

A key challenge with this approach is the absence of an inherent feature selection mechanism, making it unclear which predictors contribute most to the variability of `highhost`. Given the large number of features, the model is susceptible to the “curse of dimensionality”, where distance-based methods become less effective as the feature space grows and data becomes increasingly sparse. To mitigate this issue, we restrict our feature set to those whose coefficients did not shrink to 0 (or “.”) in the elastic-net model. This results in a reduced set of 40 features. The advantage of this approach is that feature selection is data-driven, retaining only covariates that have demonstrated predictive relevance. For categorical covariates, dummy variables are created for each level. As indicated by the elastic-net results, some dummy variables associated with the `neighbourhood.ward` feature are excluded, further reducing dimensionality.

In addition, it is essential to scale the data, as KNN relies on distance calculations. Standardisation ensures that all predictors contribute equally to the distance metric, preventing features with larger scales from dominating. The same transformations applied to the training data are consistently applied to the test data.

To tune the hyperparameter k , we perform 10-fold cross-validation over the limited search space $k = 1, \dots, 20$. As shown in Figure 4, the highest cross-validation Accuracy of 76.89%² is achieved at $k = 1$, making it the optimal choice. This is unexpected, as $k = 1$ is typically associated with high variance. Additionally, increasing $k = 2$ results in a sharp drop in Accuracy to below 73.5%, before rising above 75% at k .

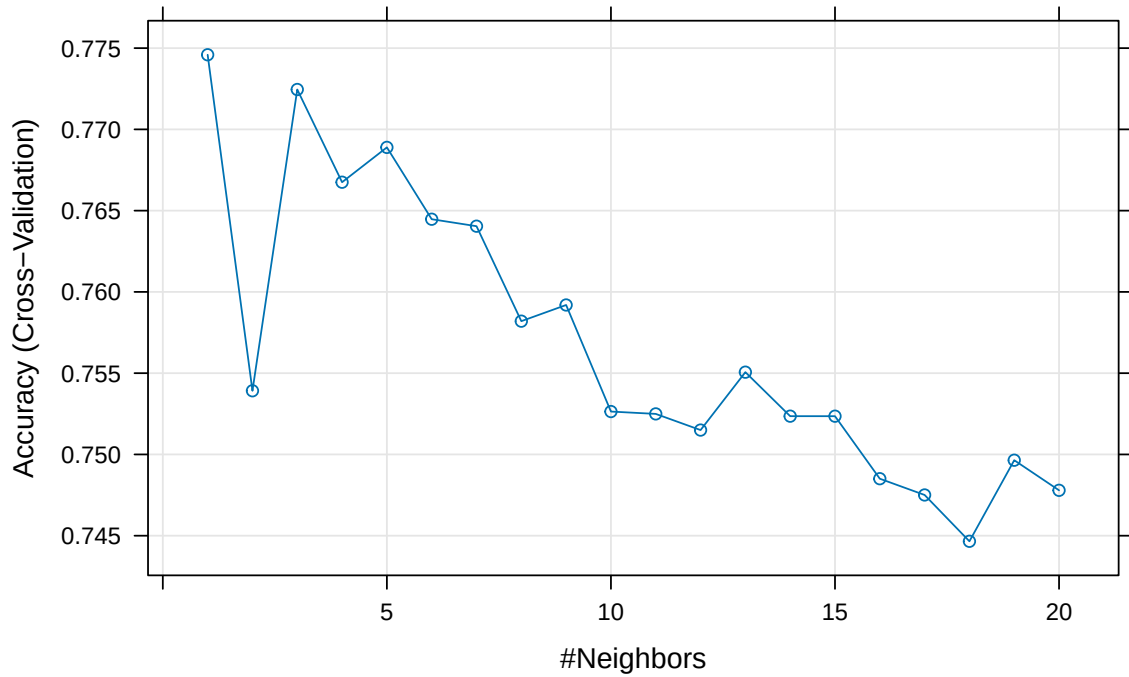


Figure 4: K Nearest Neighbours Accuracy (Cross-Validation) Plot Across the Hyperparameter Space

b) Summary of Relevant Metrics

Table of Metrics

Having fitted each of the models to the data, we now collate the different relevant metrics on Table 2 with the goal of selecting a final model in mind.

Table 2: Metrics for specified models

Model	Accuracy	F1 Score	Precision	Recall	Specificity	AUROC
Elastic-net	0.6975	0.0627	0.5549	0.0333	0.9883	0.7610
Random Forest	0.8664	0.7705	0.8077	0.7365	0.9233	0.9264
Gradient Boosted	0.8566	0.7560	0.7845	0.7295	0.9123	0.9178
K-nearest neighbours	0.7689	0.6527	0.6394	0.6665	0.8184	0.8263

Visual Illustration of Results

Figure 5 summarises the metrics from Table 2. Random forest performs best overall, with gradient boosting close behind. Elastic-net, despite high Specificity, has low F1 and Recall, indicating poor identification of **highhost**.

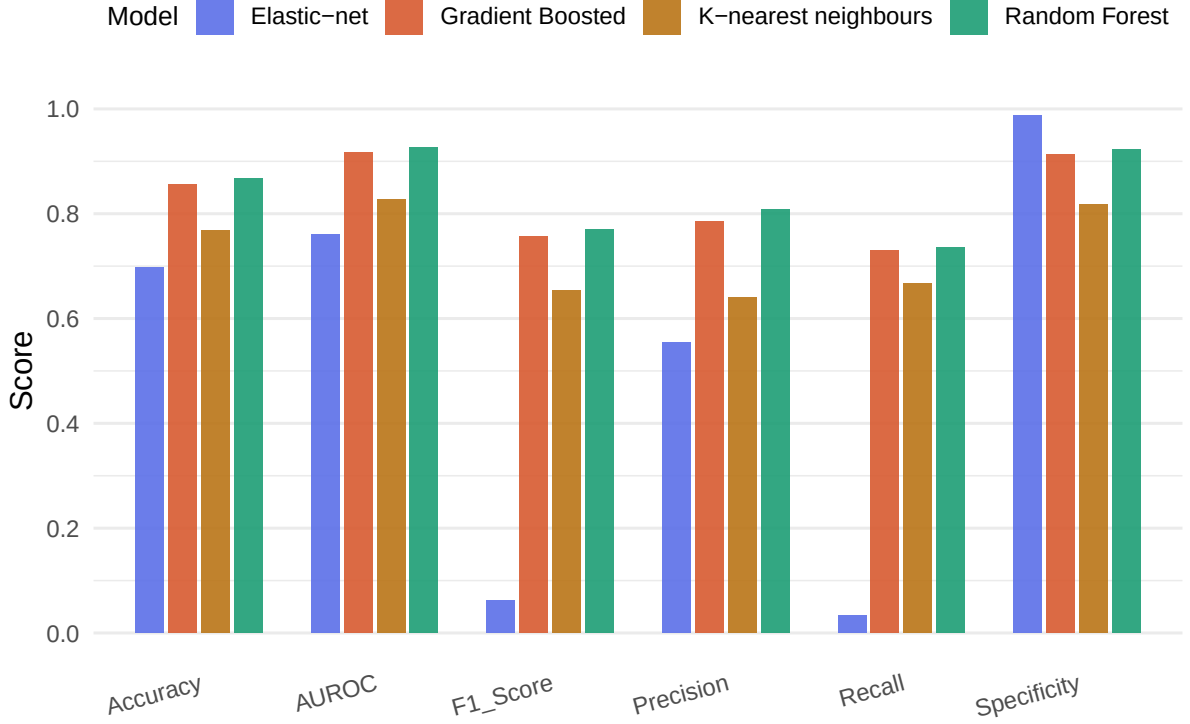


Figure 5: Model Performance Across Evaluation Metrics

c) Model Selection

The Random Forest model was selected as the final classifier based on its superior performance across the two metrics most critical to our objectives: Accuracy and Recall. Given the training data’s class distribution (a 69.5%:30.5% split between not **highhosts** and **highhosts**), a naive majority-class classifier would achieve 69.5% Accuracy. The Random Forest substantially exceeds this baseline at 86.64%², demonstrating genuine learning rather than exploitation of class imbalance.

Of greater substantive importance is Recall, given the asymmetric error costs inherent to this problem. As outlined earlier, a false negative - failing to not identify a genuine **highhost** directly undermines the study’s aim. False positives, while undesirable, carry a lower informational cost. The Random Forest achieves a Recall of 73.65%², the highest among all models evaluated, making it the most appropriate choice given our primary detection objective.

This selection is reinforced by the model’s performance on secondary metrics. Its F1 score of 77.05%² reflects an effective balance between Precision and Recall, while its AUROC of 92.64%² confirms that strong class separation is maintained across all decision thresholds, not merely the default $\tau = 0.5$. High Specificity of 92.33%² indicates that false positives are well-controlled, though this was not a primary selection criterion.

The performance of alternative models contextualises the Random Forest’s superiority. The Gradient Boosted model performs competitively, with Accuracy of 85.66%² and Recall of 72.95%², but is marginally outperformed on both by Random Forest. K-nearest neighbours lags more meaningfully, with Accuracy of 76.89%², Recall of 66.65%², and AUROC of 82.63%², indicating weaker overall discriminative capacity. Elastic-net performs poorest among the four: its Accuracy of 69.75%² is only marginally above the naive baseline (69.5%), its Recall of just 3.33%² renders it effectively unable to detect **highhosts**, and its F1 score of 6.27%² confirms that the `lambda.1se` regularisation penalty has produced a model that is too sparse to be useful in this setting. The

Random Forest thus represents the most balanced and contextually appropriate choice.

Question 3

The response variable `highhost` is by design imbalanced, and this is a deliberate trade-off to preserve the conceptual integrity of the classification. A consequence of this is that the default decision threshold of $\tau = 0.5$ is not well-suited, as it implicitly treats the two classes symmetrically in a setting where they are not. We therefore turn to threshold calibration on the already-selected Random Forest, iterating over $\tau \in [0.01, 0.99]$ in increments of 0.01 and selecting the threshold that maximises the F1 score on the cross-validated training predictions. The F1 score is the preferred criterion here because, as the harmonic mean of Precision and Recall, it penalises models that resolve class imbalance by biasing predictions toward either class.

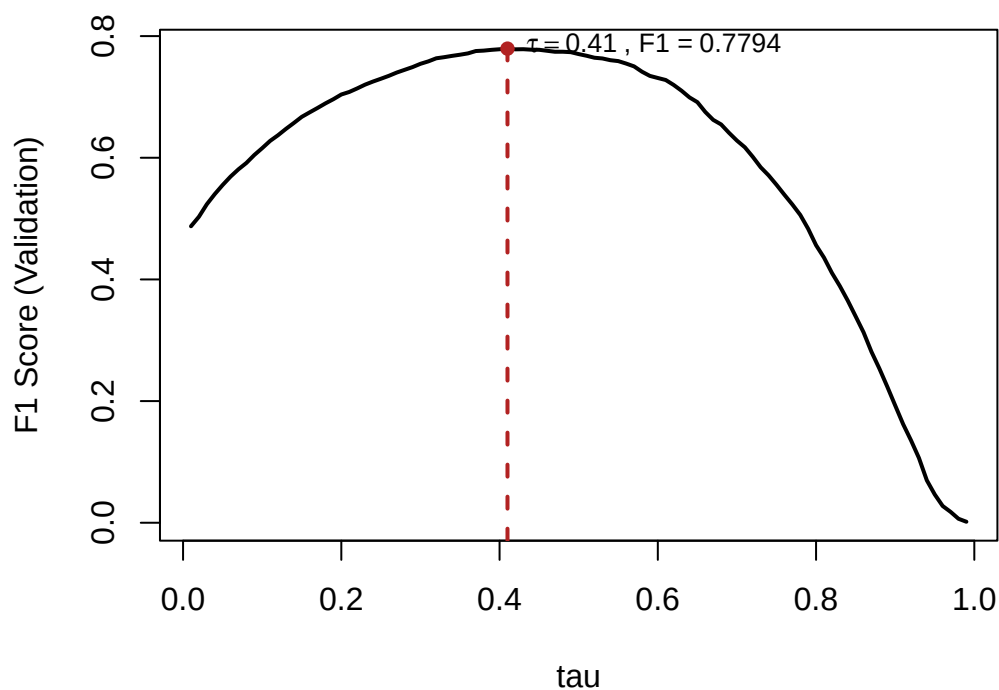


Figure 6: F1 Score Across Decision Thresholds

The optimal threshold was found to be $\tau = 0.41$, marginally below the default, reflecting the asymmetric cost structure of the problem: by lowering the decision boundary, the model is more willing to classify a host as a `highhost`. Moreover, the AUROC stands at an impressive 92.64%³.

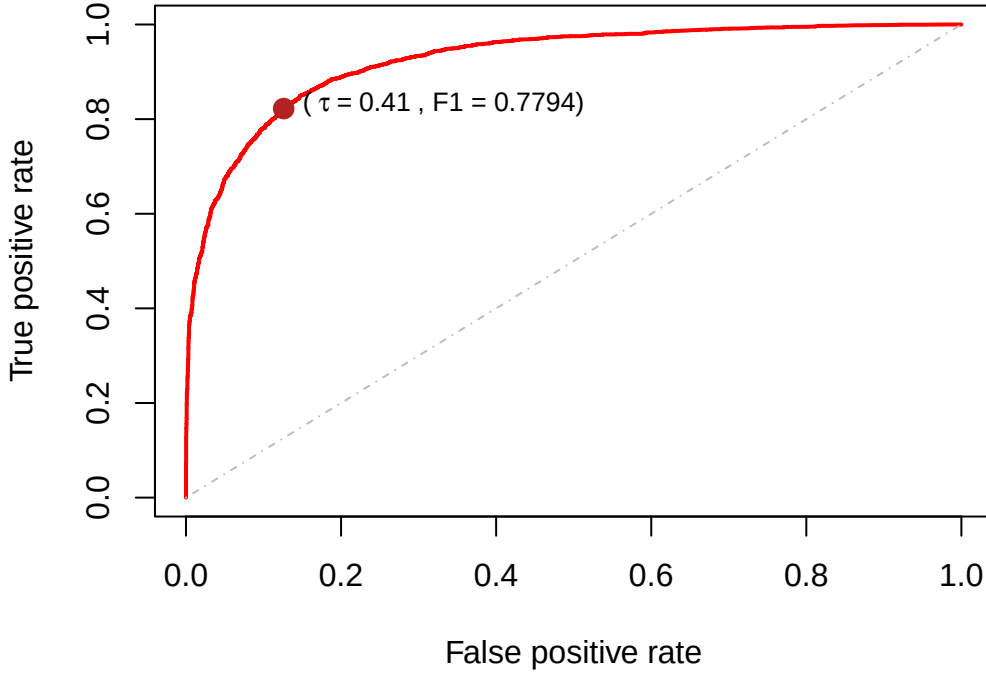


Figure 7: Validation Set ROC Curve – Random Forest

Table 3 presents the performance of this optimised decision rule ($\tau = 0.41$) across both the cross-validated training predictions and the held-out test set. While training performance is encouraging (the optimised threshold even marginally improves upon $\tau = 0.5$ on Recall; a metric of particular substantive importance in this context despite not being the direct optimisation criterion) the model generalises poorly to unseen data. Test set Accuracy of 76.70%³ represents only a 7.20 percentage point improvement over the naive majority-class baseline, a modest gain that falls well short of the model’s cross-validated performance. More concerningly, test set Recall of 55.12%³ indicates that the optimised threshold correctly identifies only approximately half of true **highhosts** on unseen data. This is a meaningful deterioration from the training Recall of 82.22%³, which suggests that the threshold calibration has overfit to the cross-validated training predictions and does not transfer reliably to the test set.

Table 3: Random Forest: Training vs Test at Optimised Threshold

Metric	Training ($\tau = 0.41$)	Test ($\tau = 0.41$)
Accuracy	0.8582	0.7670
F1	0.7794	0.5927
Precision	0.7408	0.6408
Recall	0.8222	0.5512
Specificity	0.8740	0.8628
AUROC	0.9264	0.8283

Note: τ denotes the classification threshold. The optimised threshold $\tau = 0.41$ was selected by maximising F1 on the cross-validated training predictions.

Question 4

a) Variable Importance Plots

Impurity Variable Importance

Impurity Variable importance is defined as the mean total reduction in RSS contributed by each feature across all trees, capturing how useful a variable is for splitting during tree construction rather than for out-of-sample prediction. The RSS-based criterion is known to favour continuous features over categorical or binary ones, since continuous variables provide more candidate split points and therefore more opportunities to reduce impurity. Consistent with this, the Random Forest's five highest-ranked features are all continuous - an artefact of the tree construction mechanism rather than a definitive signal of true predictive relevance.

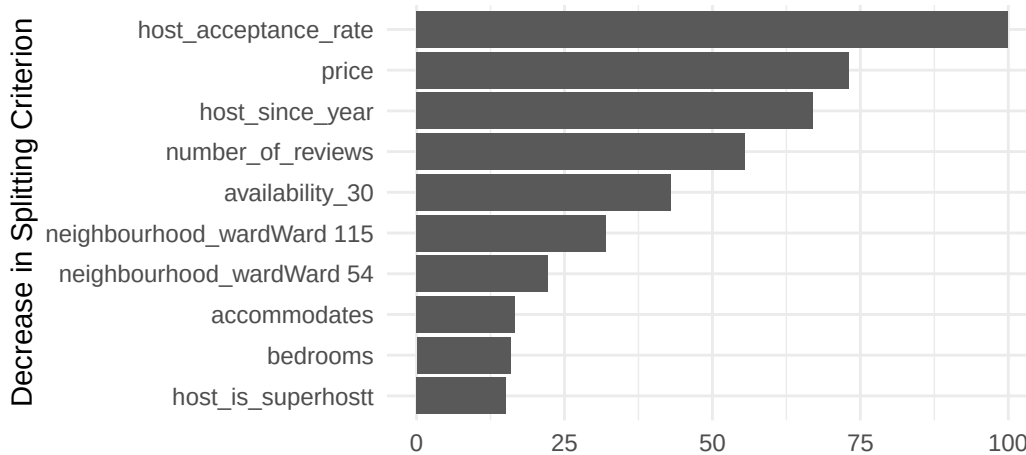


Figure 8: Impurity-Based Variable Importance Plot

From Figure 8, `host_acceptance_rate` is the most important feature for discriminating between levels of `highhost` within the tree-building process. It is followed by `price`, with `host_since_year` ranking third. The ranking of `price` is particularly illustrative of this, despite its prominence here, the earlier Elastic-net model excluded `price` via the Lasso penalty (under both the minimum and one-standard-error selections). This suggests that while `price` is effective for partitioning the feature space during tree growth, this should not be interpreted as evidence of strong predictive power in a generalisation context.

Permutation Variable Importance

Permutation importance is defined as the average increase in model misclassification error when a feature's values are randomly permuted across the out-of-bag sample, thereby severing its relationship with `highhost`. Unlike impurity-based importance, which reflects a feature's contribution to in-sample split quality, permutation importance captures the deterioration in predictive performance on unseen data — making it a more reliable proxy for out-of-sample relevance and one that is not susceptible to the continuous-feature bias that afflicts RSS-based rankings.

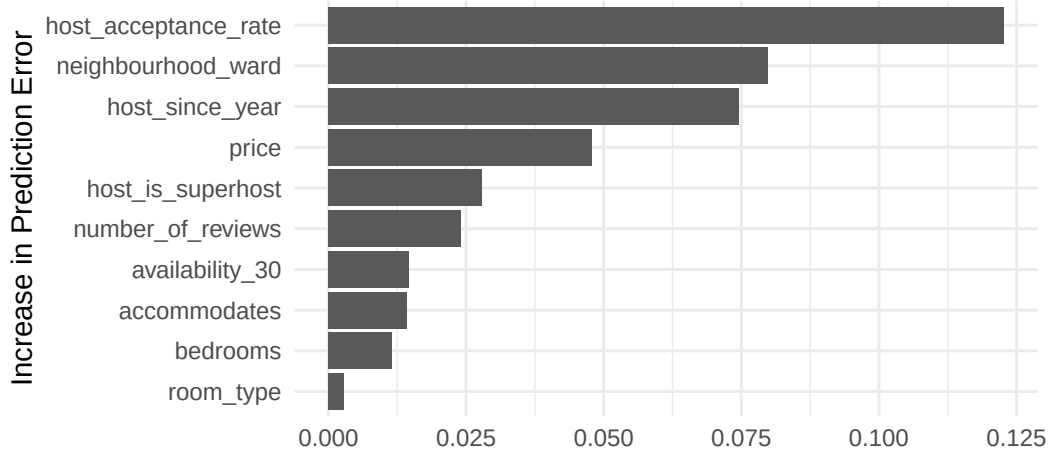


Figure 9: Permutation-Based Variable Importance Plot

From Figure 9, `host_acceptance_rate` again emerges as the most consequential feature: permuting its values produces the largest average increase in misclassification error across the ensemble. Its dominance under both importance criteria suggests that its role is not merely an artefact of the impurity-based criterion’s preference for continuous features, but reflects genuine predictive relevance. `neighbourhood_ward` ranks second, a result more aligned with the broader pattern observed across the other fitted models; this ranking carries greater substantive credibility precisely because permutation importance is not biased toward high-cardinality continuous variables. `host_since_year` again ranks third.

The fact that `host_acceptance_rate` and `host_since_year` rank highly under both criteria — despite the two measures capturing fundamentally different aspects of feature utility — strengthens the case for treating them as genuinely informative signals of large-scale host activity on the platform, rather than artefacts of the tree-growing process.

b) Partial Dependence Plots

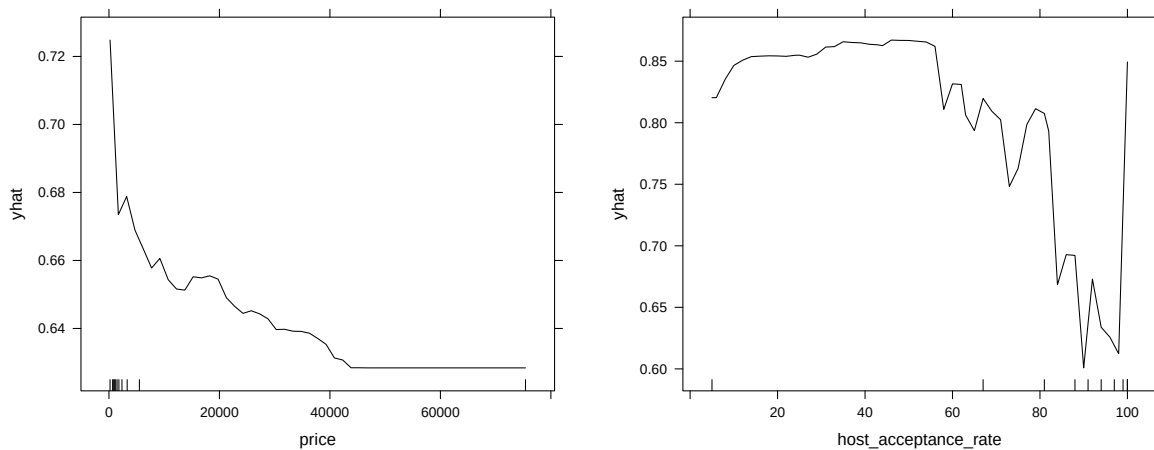


Figure 10: Partial Dependence Plots

The contradictory treatment of `price` across the modelling approaches warranted further investigation; as did `host_acceptance_rate` given its consistent top ranking across both variable importance criteria. Partial dependence plots were therefore constructed for both features.

Looking at Figure 10, the PDP for **price** reveals a negative marginal relationship: lower-priced listings are associated with a higher predicted probability of **highhost** status, with the effect plateauing at higher price values. This is consistent with the hypothesis that commercially scaled operators tend to concentrate in the cheaper, higher-volume segment of the market, managing many modestly priced listings rather than fewer premium ones. The elastic-net’s exclusion of **price** is therefore likely a consequence of its inability to capture this non-linear relationship within a linear decision boundary, rather than evidence that price is genuinely uninformative.

In contrast, the PDP¹⁰ for **host_acceptance_rate** exhibits a non-monotonic pattern. The predicted probability of **highhost** status peaks at intermediate acceptance rates, declines at very high levels, and remains elevated at very low levels. The high probability at elevated acceptance rates is intuitive: large-scale operators are likely to run more systematic, responsive operations. The elevated probability at low acceptance rates is less obvious, but may reflect a subset of high hosts who operate at scale and yet remain selective or slow to respond, potentially due to capacity constraints in managing large portfolios.

c) Subsample of False-Positives and False-Negatives

Table 4 and Table 5 present five randomly sampled false positives and false negatives respectively from the test set predictions of the optimised Random Forest. Several patterns are discernible across both error types.

All five false positives are entire home/apt listings, suggesting room type is a strong but insufficient signal of commercial operation. Two record high prices (R6,558 and R6,552), contrary to the negative price-**highhost** relationship identified in the PDP, while low **availability_30** values mimic the booking patterns of commercial operators. Ward 115 appears twice, consistent with high-demand wards inflating predicted **highhost** probabilities.

Among the false negatives, the most notable observation is an extreme price of R26,000, which the PDP associates with low **highhost** probability, misleading the model. Three observations record zero **availability_30**, suppressing predicted commercial operation probabilities despite true **highhost** status. Across both error types, misclassification tends to involve entire home/apt listings in high-demand wards with atypical pricing or availability — features that serve as imperfect proxies for host scale in isolation.

Table 4: Five Randomly Sampled False Positives from Test Set Predictions

True Class	Year Joined	Acceptance Rate (%)	Superhost	Ward	Room Type	Acc.	Beds	Price (R)	Avail. (30d)	Reviews	Pred.
0	2016	88	No	Ward 54	Entire home/apt	6	3	6558	9	22	1
0	2020	77	No	Ward 30	Entire home/apt	2	0	317	30	7	1
0	2012	94	No	Ward 115	Entire home/apt	7	4	6552	7	21	1
0	2022	84	Yes	Ward 115	Entire home/apt	4	2	2018	12	28	1
0	2017	95	No	Ward 115	Entire home/apt	2	1	830	0	25	1

Note: Pred. = Predicted class. Acc. = Accommodates. Avail. = Availability. All observations were predicted as **highhost** (1) but belong to the low host class (0).

Table 5: Five Randomly Sampled False Negatives from Test Set Predictions

True Class	Year Joined	Acceptance Rate (%)	Superhost	Ward	Room Type	Acc.	Beds	Price (R)	Avail. (30d)	Reviews	Pred.
1	2019	94	Yes	Ward 115	Entire home/apt	2	1	1174	16	110	0
1	2018	100	No	Ward 113	Entire home/apt	4	2	604	10	26	0
1	2013	76	Yes	Ward 74	Entire home/apt	10	5	26000	0	1	0
1	2015	93	No	Ward 74	Entire home/apt	2	1	1080	0	27	0
1	2021	94	No	Ward 77	Entire home/apt	2	1	646	0	7	0

Note: Pred. = Predicted class. Acc. = Accommodates. Avail. = Availability. All observations are true **highhosts** (1) that were incorrectly predicted as low hosts (0).

References

Moodley, Doubell and Evans (2025) Opinion: Cape Town now has 26,000 Airbnbs – but what are the rules? *News24*, 21 September 2025. Available at: <https://www.news24.com/business/opinion/opinion-cape-town-now-has-26-000-airbnbs-but-what-are-the-rules-20250921-0671> [Accessed: 23 March 2026]